

01.00 Please Go On: An Introduction

MY
USER:
FATHER.

01.01	Recovering ELIZA's Code
-------	-------------------------

01.02	The "Eliza Effect"
-------	--------------------

01.03	Systems of Promise and Peril
-------	------------------------------

01.04	Re-Creating ELIZA
-------	-------------------

01.05	About This Book
-------	-----------------

DOCTOR: YOUR FATHER

When computers were still room-sized machines, they did not speak or listen. People programmed these giants using paper punch cards and physical typewriter keyboards—there were no screens, no touchpads, not even mice. Computers were used to speed up calculations, manage payroll, automate banking systems, and detect ballistic missiles, among other applications. Although, for centuries,¹ ideas of thinking machines and automata have been, by turns, cautionary tales, fantasies of control, and utopian dreams, the expectations for computation typically far outpaced the reality—from Asimov’s three laws of robotics to *The Jetsons*. But at the time the thought of conversing with such a machine would have seemed as awkward as chatting with a toaster.²

When it debuted in the mid-1960s, the ELIZA software program transformed the way people thought about interacting with computers. As the first chatbot,³ ELIZA demonstrated how a calculation machine might engage in conversation, ushering in a host of social and technical questions that still resonate today. Now we don’t think twice about interacting with a machine in real time, conversing over text, or even speaking into the air to ask about the weather. In many ways, ELIZA shaped not only the way we think about interacting with computers, but also how we *think about them*. It began to give a reality to the science fiction stories of how we expect computers to work. Although ELIZA was far from a faultless conversation partner, it astonished its users. ELIZA expanded the new technical imaginary, gesturing toward a world to come where we might work *with* computers rather than *on* computers.⁴

ELIZA did not just spark excitement, it also triggered alarm at the trust humans all too easily place in computers that respond to them in a conversational way. This caused the developer of ELIZA, Joseph Weizenbaum, a professor at Massachusetts Institute of Technology (MIT), to think very seriously about how our inclinations towards companionship, social trust, and community could be manipulated by computer interfaces. As he would write repeatedly over the following decades, computers do not understand but only calculate. Computers have no sense of morality, no sense of justice or rights, and they certainly are unable to take on a responsibility toward others. Though they might appear to be intelligent, they are not. Though they might appear to be sympathetic, they are not. Instead, they are instrumental, technical, and calculating, not just about mathematics or computing but about everything, including human life, feelings, and communities. Weizenbaum foresaw correctly that the development of conversational systems would likely result in a severe clash of values between the safeguards of social norms on the one side and, on the other, the priorities of a technocratic class using computers to control and manipulate people.

However, in the sixty years that ELIZA has been influencing computation and culture, popular histories have portrayed it primarily as a software program that could converse as an automated psychologist, modeled on psychotherapy techniques designed by Carl Rogers. ELIZA uses simple pattern matching and script-based responses to simulate a

therapist's conversation. In the conventional account, it is the first of what we now call chatbots and is known for "fooling" even the secretary who watched Weizenbaum create it. That's how the story goes. However, in all these accounts, one essential piece has been missing: the ELIZA source code. After a deep dive in the MIT Archives, we have found that source code and it tells a much more complex story. This book represents a major research project into ELIZA as we recover, close read, and discuss ELIZA's source code for the first time.

By examining the program's code, dialogues, and paratexts, *Inventing ELIZA* seeks to challenge the popular story of ELIZA, instead revealing that ELIZA is, in fact, multiple ELIZAs, developed as different versions with differing functionalities, designed to run a wide variety of scripts or personas—not only DOCTOR—and built using a series of technical innovations (like abstraction, API-like software layering, list-processing language extensions, real-time interaction, and scripting functions). We seek to correct and to complicate ELIZA's history and influence by exploring the misconceptions, multiple versions, and missing code of ELIZA. We question a computing history that has lost many of its nuances, along with the artifacts that document these differences. Using the methods of critical code studies, we close read the historic source code as a collaborative media archeology project. We chose this software not just for its technological achievements, which were groundbreaking, but also because ELIZA speaks to this moment when conversational interfaces have returned into public use at a greater intensity and with much more impressive capacities—marked most dramatically by the appearance of tools like ChatGPT. Such systems offer new possibilities for conversational computing and its incorporation into a variety of technological devices and social situations, but their proliferation also calls for the same cautions Weizenbaum foresaw. Our journey into the code and its history has brought us into dialogue with Weizenbaum as an innovator who also saw the impact of his work as a warning. *Inventing ELIZA* is dedicated to understanding how ELIZA works, how and why it was written, what it has meant, and how it continues to influence culture.

ELIZA has set a template for human-computer conversations to come.⁵ Its significance lies both in its technical innovation and also in its ability to capture the public imagination and to shape expectations for artificial intelligence. The development of ELIZA's language-processing capabilities and its DOCTOR script's conversational interactions established design patterns that would persist through decades of technological development. ELIZA/DOCTOR adds a critical chapter to the history of computing—including through its entanglements with problematic assumptions of gender, race and ethnicity, and class, as well as philosophical questions about what it means to be human. From Amazon's Alexa to ChatGPT, from HAL in *2001: A Space Odyssey* to Samantha in *Her*, the contemporary vision of conversational AI traces back to those early interactions between people and ELIZA/DOCTOR in Weizenbaum's MIT lab.

01.01 Recovering ELIZA's Code

To appreciate ELIZA's powerful techniques operating in just a few hundred lines of code, imagine the state of computing in the mid-1960s (see chapter 4). These large machines relied upon magnetic tape and punch cards. Weizenbaum's version of DOCTOR was, as Nick Montfort describes, "printed more or less permanently on paper, rather than transiently appearing on the screen, [which might] influence how we understand the system as literary and as psychotherapeutic. A session that leaves a printed record, like a diary, may be experienced differently than a transient on-screen encounter."⁶ Today, we are more familiar with ELIZA/DOCTOR on screens, but these printouts are crucial for appreciating the material specificity of what it would have been like to use ELIZA in the 1960s. The computer ELIZA was developed on, an IBM 7094 running Compatible Time-Sharing System (CTSS),⁷ did not support mixed upper and lower case letters, unless you were using special libraries.⁸ How did these tangible aspects affect users' experience of ELIZA, when they were not immersed in the same kinds of digital environments we are today? Using ELIZA would have been their first experience chatting with a program, with no prior context to draw on. To imagine how different their expectations of chatting with DOCTOR would have been at that early moment, we rely on material traces from the archives.

Our discovery in 2021 of the lost ELIZA source code in the MIT Archives' Distinctive Collections revealed surprising details that transformed our understanding of ELIZA. (For example, in early printouts of the source code, it appears that the program is called SPEAK before it is ever referred to as ELIZA!)⁹ This inspired our international, interdisciplinary research team to convene for four years, unravelling the mysteries of the code. It also meant (re)investigating the larger historical context surrounding ELIZA to rewrite the stories, not only of this code, but also of how it has shaped the histories and futures of human-computer interaction. Nevertheless, this work has not progressed along a straightforward path, and the revised stories of ELIZA we present here are not stable, but plural, iterative, and endlessly reinventing themselves.

Despite ELIZA's considerable impact on computer science and culture, with versions of its program widely run and referenced, the original code was never published nor distributed. In his most widely read paper, published in 1966 in the *Communications of the Association of Computing Machinery (CACM)*,¹⁰ Weizenbaum describes in detail the workings of ELIZA and the DOCTOR script. His description has been used to create the many versions of ELIZA around today—all made without access to the actual source code. Even with the many copies of ELIZA, no one knew exactly how the original ELIZA did what Weizenbaum described. The missing code did not suppress appetites for ELIZA-like systems, but it left a large gap in understanding the original program as well as its impact and legacy.

Our investigations in the MIT Archives found one complete version of Weizenbaum's ELIZA code and DOCTOR script. Our comparative

analyses reveal that this code is close to but not the exact version that Weizenbaum detailed in his article. Among Weizenbaum's papers, we also found evidence of multiple other versions of the ELIZA program; multiple versions of published dialogues, some of which had been hand-edited; additional unpublished DOCTOR dialogues; as well as previously unknown scripts that were entirely distinct from DOCTOR. With these findings, the notion of a definitive ELIZA and its familiar distinctive DOCTOR persona began to disintegrate into a proliferation of ELIZAs.

The discovery and archaeology of the original ELIZA source code represents a significant intervention in the history of computing. By examining the actual MAD-SLIP¹¹ implementation of ELIZA rather than relying on later reconstructions and reimplementations, we challenge taken-for-granted assumptions about this key software artifact. For example, importantly, the source code reveals that ELIZA was not merely a simple pattern-matching chatbot but can be better understood as a sophisticated platform designed for multiple "personas," or scripts, with a complex set of capabilities, including script editing and contextual memory. This transforms our understanding of early AI development by demonstrating that Weizenbaum's technical innovations were far more advanced than previously documented.¹² (ELIZA is even Turing complete, see chapter 7.) Moreover, the discrepancies between his published descriptions and the actual implementation help to show the gap between theoretical computational models and their material instantiations in computer source code, a tension that continues to shape digital culture today. By recovering this code, we reposition ELIZA in the genealogy of computational media, not just as a historical curiosity but instead as a complex technical achievement. *Inventing ELIZA* is in fact a complex story of many ELIZAs.¹³

Contrary to another popular misconception, ELIZA and DOCTOR are not one and the same. The name ELIZA refers to the entire system Weizenbaum built, and the DOCTOR script is just one of several scripts written for that system. ELIZA is an interpreter, a program that interprets a particular form of data called a script.¹⁴ The most famous ELIZA script, DOCTOR, inspired numerous unauthorized copies, reimplementations, and confusions. It produced versions that often omitted the behavioral element of the original ELIZA software. This omission was unfortunate—as it ignored both the additional scripts as well as ELIZA's potential for multiple personas and flexibility.

These adaptations implement the DOCTOR script as if it *were* ELIZA, rather than implementing ELIZA as a system for running DOCTOR along with multiple other chatbot personas. For simplicity, and in keeping with their own naming conventions, we will call these adaptations "ELIZA" even though they are technically adaptations of DOCTOR. Versions can be found in early programming languages as well as in most contemporary high-level languages, from Python to JavaScript. Shortly after the publication of Weizenbaum's 1966 paper, Bernie Cosell programmed an ELIZA in Lisp (1966, 1969, 1972), based on Weizenbaum's published

algorithm (see chapter 8).¹⁵ Lisp was rapidly becoming the primary language of AI, and Weizenbaum's DOCTOR script is formatted similar to a Lisp S-expression (symbolic expression). This contributed to the persistent misconception that ELIZA was originally written in Lisp, although one need only read the first page of the *CACM* paper to learn that ELIZA was actually written in the language MAD-SLIP. In 1977, Jeff Shrager's BASIC ELIZA was printed in *Creative Computing* magazine, re-creating ELIZA for personal computers.¹⁶ As a result of these coincidences, the ELIZA known in academic communities was the Lisp version, and the one known to the public was the BASIC version.

ELIZA's original code went virtually unseen for almost 60 years. Written in just over 400 lines of code, the ELIZA program is surprisingly sophisticated. When people conflate ELIZA and DOCTOR, they mistake the DOCTOR script to be the sole aim of Weizenbaum's research, when in fact he offered DOCTOR merely as an illustration of what the system could do. By choosing to simulate (or, in his words, "parody"¹⁷) Rogerian nondirective psychotherapy using computation, Weizenbaum selected a premise that allowed ELIZA to operate with very little context while remaining a plausible conversationalist. This reflexive style of therapy mirrors back the response of the patient to elicit more details. This premise meant that ELIZA could create the illusion of a conversational partner who kept previous comments in mind by storing and recalling certain phrases made by the user as the conversation progressed.

What makes interacting with ELIZA/DOCTOR so enticing is the pattern-matching and substitution algorithm at its core. But what makes this algorithm come to life are Weizenbaum's selection of the psychotherapist persona and his code's elegance and concision in responding to the challenges of producing coherent conversation and seemingly relevant phrase recall. In the *CACM* article, Weizenbaum describes how simple transformations can create the impression of "understanding" complex input: "an input sentence is scanned from left to right. Each word is looked up in a dictionary of keywords. If a word is identified as a keyword, then (apart from the issue of precedence of keywords) only decomposition rules containing that keyword need to be tried."¹⁸ The system responded to any input with a typed response in English, and there is also mention of German and Welsh versions. The algorithm replaces pronouns from first person to second person (and vice versa) and simulates understanding. However, lacking the code itself, misconceptions about how ELIZA produced these simulations sprang up over the years.¹⁹

In fact, the instructions Weizenbaum gives in the article could not have output the oft-reproduced conversation that the article gives as an example, *but the code we have found can*. Indeed, essential elements of the program that were not described in the article have been located in the recovered code. These reveal new aspects of ELIZA that this book connects to the program's legacy, insights that also allowed us to create a reconstruction of ELIZA that replicates the original 1966 version's dialogue exactly.²⁰

In one key insight, our exploration of the ELIZA code revealed the precise workings of “a certain counting mechanism” (see chapter 3.2) that makes this essential memory recall possible. Although Weizenbaum described the mechanism that stores and recalls users’ statements in memory, his article’s description glossed over two important details about how the memories are selected and recalled, which the study of the code reveals, allowing for the exact recreation of responses the original ELIZA would have made.

In another example, Weizenbaum’s article describes two delimiters for the end of clauses—“,” and “.”—that discard any additional text past the delimiter once a keyword is found in a phrase. A delimiter is a symbol that tells the computer that a break can be made in the input text. However, both the ELIZA code we uncovered and the sample dialogue he published in that article use a third delimiter as well—the word “but.” Consider the following exchange from the article:

You are not very aggressive but I think you don’t want me to notice that.
 WHAT MAKES YOU THINK I AM NOT VERY AGGRESSIVE

This response would not be possible unless “but” is a delimiter. If it were not a delimiter, that response would be, “WHAT MAKES YOU THINK I AM NOT VERY AGGRESSIVE BUT YOU THINK I DON’T WANT YOU TO NOTICE THAT.” Without the “but” delimiter, the part of the phrase following the word “but” is no longer discarded but is instead included in the transformed output. The discovery of this discrepancy between the code and the article highlights the ways code can stay in flux as programmers revise, extend, and rethink it.

The addition of the “but” delimiter is just one sign that ELIZA was being actively developed over time. In many ways ELIZA should be seen as a process, rather than a product, of software programming—although we can identify specific milestones in its development as major versions of the software (see below). Weizenbaum described how he continued work on ELIZA across multiple versions, as “difficulty with the [original ELIZA] system currently [meant] that it [could] do very little other than generate plausible responses.” Weizenbaum explained:

The [new version of] ELIZA differs from the old one in two main respects. First, it contains an evaluator capable of accepting expressions (programs) of unlimited complexity and evaluating (executing) them. It is, of course, also capable of storing the results of such evaluations for subsequent retrieval and use. Secondly, the idea of the script has been generalized so that now it is possible for the program to contain three different scripts simultaneously and to fetch new scripts from among an unlimited supply stored on a disk storage unit, intercommunication among coexisting scripts is also possible.²¹

These later advances were, sadly, poorly documented and their source code remains unrecovered, as do many of the scripts of the other personas.

However, in chapter 5, we include examples from some of the recovered scripts, showcasing ELIZA's alternative personas. As Weizenbaum notes: the most famous script is the one where ELIZA plays the psychiatrist, but there were others. I remember an undergraduate at that time who is now full professor at MIT—Steve Ward by name—who wrote a script that was a parody of the way President Nixon talked. That was not hard to do: Nixon has a very characteristic way of speaking, like “Let me say this about that” and so on.²²

Although this is often forgotten, Weizenbaum described ELIZA as a family of programs.²³ Most people today assume there was only one version of ELIZA, but our research shows that there were at least five major versions of the ELIZA program (there may be many more). This includes the versions discovered in the archive: a flowchart and outline (1965a), and the complete source code (1965b). ELIZA (1966) is the version described in Weizenbaum's CACM paper. This ELIZA was developed by Weizenbaum during the early 1960s, and it is this version (and its offshoots) that is commonly known as the world's first chatbot. There were also later versions we call ELIZA (1967, 1968+) that had much more sophisticated scripting functions using the language OPL (see table 1.1 below).

As Weizenbaum iterated the system, ELIZA found its way into new fields of research. ELIZA was built under the auspices of MIT's Project MAC, which was funded by the Department of Defense's Advanced Research and Projects Agency (ARPA, now DARPA), with MIT producing annual reports detailing the experiments conducted with the system. According to Project MAC Progress Report II (1964–65), ELIZA was used “for experiments on two-person conversational interaction at the Massachusetts General Hospital,” whereas Report III (1965–66) describes ELIZA's use in a “study designed to assess the phenomenon of misunderstanding in man-machine natural-language communication, and establish the conditions and limits under which natural-language communication between man and machine could be made plausible.” These project reports emphasize ELIZA's original conception as an educational and research tool rather than merely a conversational chatbot.²⁴

ELIZA (1967) added script handling²⁵ and could run other scripts with names like ARITHM, F29, and FIGURE (see table 1.2). A colleague in the Education Research Center at MIT, Edwin F. Taylor, then built on these versions to develop ELIZA (1968+). This version was greatly extended beyond ELIZA (1966), and was used at MIT and Harvard to build Computer Assisted Instruction (CAI) systems. Each of the versions we have identified are shown in table 1.1. That Weizenbaum called all of these programs ELIZA, in the singular, can be confusing (not least for the authors of this book). Surely, the history of chatbots may have been quite different if the world had seen the code for each of these versions and if the versions had been clearly delineated. As David M. Berry suggested in a Critical Code Studies Working Group on the ELIZA code, it became

clearer that not only are there multiple ELIZAs but that this has an important bearing on its identity as a research object. ...

ELIZA is certainly a work in progress, less than a finished software object, and very much like a process of research activity with stop-offs along the way. ... [So] we are looking at both a technical object, a cultural object and a historical object and that these things might not always be the same thing.²⁶

As ELIZA merged with DOCTOR to become a cultural object, the programs we group under the name ELIZA have been obscured.

Table 1.1 Comparison of ELIZA versions.

Version	Delimit “.”	Delimit “,”	Delimit “but”	NEWKEY Function	Script Edit Function	Keyword Stack	Hardcoded Messages	Script Handling	Evidence
ELIZA 1965a	Yes	Yes	—	—	—	—	—	Script	MIT Archive flowcharts
ELIZA 1965b	Yes	Yes	Yes	—	CHANGE Function	—	Yes	Script	MIT Archive Source Code
ELIZA 1966	Yes	Yes	Yes	Yes	EDIT Function	Yes	—	Script	Weizen- baum, “Eliza”
ELIZA 1967	Yes	Yes	Yes	Yes	EDIT Function	Yes	—	OPL ²⁷	Weizen- baum “Eliza”; Taylor, “Auto- mated Tutoring and Its Discon- tents.”
ELIZA 1968+	Yes	Yes	Yes	Yes	EDIT Function	Yes	—	OPL	Taylor, “Auto- mated Tutoring and Its Discon- tents.”

- 1 ELIZA 1965a: Delimits sentences with only “.” and “,” as shown in MIT archive flowcharts.
- 2 ELIZA 1965b: Version recovered from MIT Archives. Delimits sentences with “.” and “,” and “but.” Lacks NEWKEY function. Includes undocumented CHANGE function and hard-coded messages.
- 3 ELIZA 1966 CACM: Functionality includes the NEWKEY function and keyword stack, as described in Weizenbaum, “ELIZA,” as well as the “but” delimiter, which is omitted from the article but present in the dialogues.
- 4 ELIZA 1967: Adds sophisticated script handling using OPL, as described in Weizenbaum, “Contextual Understanding by Computers,” “OPL-I,” and Taylor, “Automated Tutoring and Its Discontents.” Also evidenced by the extant ARITHM, F29, FIGURE scripts in the archive.
- 5 ELIZA 1968+: A description of the planning of this version appears in Taylor, “Automated Tutoring and Its Discontents.” The scripts SPACKS, INTRVW, and FVP1 also give evidence of its use in work developed by Walter E. Daniels.

As part of our version mapping, we discovered a section of the code (the CHANGE function) that gives ELIZA extra functionality that Weizenbaum called “teachable” This means it provides a method for a user to revise and extend scripts with new rules for text transformations while the program is running. In subsequent versions of ELIZA, this function is replaced

by an EDIT function that calls an operating system editing program named ED (we discuss this in more detail in the ELIZA code annotations, line 10). Weizenbaum alludes to this functionality briefly when describing ELIZA: “Its name was chosen to emphasize that it may be incrementally improved by its users, since its language abilities may be continually improved by a ‘teacher’. Like the Eliza of *Pygmalion* fame, it can be made to appear even more civilized, the relation of appearance to reality, however, remaining in the domain of the playwright.”²⁸ Indeed, Weizenbaum saw programming as an experimental practice: “One programs, just as one writes, not because one understands, but in order to come to understand. Programming is an act of design.”²⁹ He valued ELIZA as an evolving work, meant to enable one to understand human-computer interaction through language.³⁰ That ELIZA is not only a system for engaging conversational agents—it is also a platform for writing and editing scripts—is one of the many aspects of the program lost in the popular ELIZA imaginary.

Even programming languages themselves had, and have, material specificity—the specific hardware mattered particularly in this historical period. In a letter to the editors of the *CACM*, Weizenbaum also said of the scripting language he wrote, “SLIP is entirely host language-dependent. A FORTRAN-based SLIP will look quite different from a MAD-based SLIP, for example. A given host language is in general also dependent on the machine for which it is implemented and even, in some cases, on the monitor system within which that language is exercised.”³¹ By challenging ideas of ELIZA as a single program, a single chatbot persona, a single legacy, we can expand the many complex narratives and entanglements of ELIZA.

01.02 The “Eliza Effect”

The dialogue that opens this chapter is an excerpt of a published conversation purported to be between a young woman and DOCTOR. That dialogue has been reprinted countless times and has inspired programmers and writers to dream up many of the chatbots that followed. Yet the closer one inspects that dialogue, the more questions arise: Who was this young woman? Was she a real person, or is she Weizenbaum’s invention? How exactly did the ELIZA system generate its responses, and how much were they edited? Why did the system work so well to draw people in?

ELIZA/DOCTOR helped catalyze a mode of thought and an anxiety about our relationships with computers. Weizenbaum explored this in his 1976 book *Computer Power and Human Reason*, invoking philosophical, social, and political critiques. The unique machine interaction presented by his program revealed how new forms of human-computer relation would have profound effects that he attempted to explore and to contest. After seeing its public reception, Weizenbaum was startled by the quick and often emotional attachments people would form with ELIZA, which he saw as “clear evidence that people were conversing with the computer as if it were a person who could be appropriately and usefully

addressed in intimate terms.”³² The tendency to attribute empathy and invest private feelings into a computer puzzled Weizenbaum. He was concerned by the extent to which people associated rationality with computation, and ascribed understanding and intelligence to computer systems where none existed. This tendency became known as the “ELIZA effect.” The term appeared in online forums by 1991 but its use predated that appearance by decades.³³ Sociologist Sherry Turkle defines “the Eliza effect” as “our more general tendency to treat responsive computer programs as more intelligent than they really are. Very small amounts of interactivity cause us to project our own complexity onto the undeserving object.”³⁴ Cognitive and computer scientist Douglas Hofstadter describes it as “the susceptibility of people to read far more understanding than is warranted into strings of symbols—especially words—strung together by computers,”³⁵ which applies easily to generative AI systems today.

To understand the power and provocation of ELIZA, we can look to the infamous challenge formulated by computer scientist Alan Turing in the essay “Computing Machinery and Intelligence,” in which Turing posed the question, “Can Machines Think?” Turing premised his thought experiment on a parlor game—not about technology but about gender: A man and a woman are hidden behind a curtain and an interrogator tries to identify who is which gender by asking a series of questions. The man tries to mislead the interrogator, pretending to be a woman, while the woman tries to convince the interrogator of the “correct” answer. That is, both of them claim they are the “real” woman, a challenge to essentialist notions of gender.

In his revision of this game, Turing replaces the original gender question with what is now called the Turing test.³⁶ In this challenge a machine pretends to be a man rather than a man pretending to be a woman. More than a mere opening gambit, Turing’s choice of the initial gender imitation game has ensured that artificial intelligence would remain intertwined with questions of gender and identity. In this sense imitation, drag, and gender deconstruction laid the groundwork for AI and the performance of intellect. Weizenbaum’s ELIZA picks up where Turing left off, not least with the opening line of the dialogue: “Men are all alike.”³⁷

Nonetheless, even though Weizenbaum references Turing’s imitation game in his 1966 paper introducing ELIZA, he explicitly distances his creation from any claims of intelligence:

The crucial test of understanding ... is not the subject’s ability to continue a conversation, but to draw valid conclusions. ... In order for a computer program to be able to do that, it must at least have the capacity to store selected parts of its inputs. ELIZA throws away [most] of its inputs. ... ELIZA in its use so far has had as one of its principal objectives the *concealment* of its lack of understanding.³⁸

This assessment shows that ELIZA was never intended to pass the Turing test, but rather to explore the psychological factors that might lead humans to misinterpret its capabilities.³⁹

In that tradition, ELIZA continues to play with the notion of *performative identity*, as a vehicle for personas made out of responses that

stick to the script, literally and figuratively. By naming the system after Eliza Doolittle—a working class female character who is taught to pass as an upper-class white woman—Weizenbaum continues Turing’s provocation about the performance of identity. “I chose the name ‘Eliza,’” Weizenbaum says, “because, like G.B. Shaw’s Eliza Doolittle of *Pygmalion* fame, the program could be taught to ‘speak’ increasingly well, although, also like Miss Doolittle, it was never quite clear whether or not it became smarter.”⁴⁰ As Shaw’s Eliza performs race and ethnicity, class, sexuality, and gender through linguistic transformation, the ELIZA system performs an assumed persona through scripted and repetitive linguistic patterns and transformations, but without possessing humanlike understanding. Feminist philosopher Judith Butler’s theories of gender performativity offer a possible framework to think about this.⁴¹ Butler argues that gender and sexuality are not innate but are performed through repeated acts. They are iterated.⁴² Just as Eliza Doolittle challenges assumptions of class by performing speech that help her pass as among the upper classes,⁴³ Weizenbaum’s system performs personas like DOCTOR through speech acts, or code acts, and sample dialogues that perform gendered, classed, and racialized identities.

In this way, we argue that ELIZA both reinforces and challenges assumptions about gendered and embodied communication. It is interesting to note that in the published dialogues and popular stories about ELIZA, the women who appear alongside the program remain unnamed. They are conversing with a therapist called DOCTOR, a name that, though it does not carry gender markers today, would have sounded like a man’s title in the 1960s. Therefore, stories that suggest that these women confessed their secrets to this artificial doctor carry a gendered message, including the fantasy that one can be disembodied.⁴⁴ In this way, the ELIZA system becomes a site where questions of identity, performativity, and embodiment play out across historical algorithms in ways that we believe have implications for contemporary AI systems.⁴⁵

01.03 Systems of Promise and Peril

As norms, values, and ideology become embedded within algorithmic forms, exploring details of a software object like ELIZA reveals culturally and historically specific assumptions about what software can and should be, including ideas about what technology is for and how it has developed. While ELIZA might look unsophisticated to a contemporary audience, it was already addressing, in the 1960s, many of the design questions that persist in the systems we use today. Fundamentally, these systems ask how humans and machines should interact, by what means communication can be represented computationally, and to what extent machines can and should be imbued with an ability to influence users.

ELIZA is a common touchpoint not only because it was one of the first chatbots and helped launch a field of computational agents but also

because it intersects with so many of the computing innovations that followed. Alongside developments for string processing and text analysis, it influenced research on text synthesis, entity recognition, and sentiment analysis. It emerged alongside research in machine translation, semantic networks, speech recognition, speech synthesis, and other techniques that developed into the group of tasks we now call “natural language processing” (NLP), the area of computation that deals with how computers can parse, interact with, process, and output languages that are used by people as opposed to programming languages used by computers. In practice, these tasks are often combined to create automated agents and many other kinds of systems.

The reappearance of ELIZA-like chatbot interfaces in contemporary large language models shows that early software genealogies can help us understand new technologies. ELIZA is a useful foil for emerging models because—though much has changed, much more remains the same. The history of NLP emerges from and overlaps the era of ELIZA and early AI work at institutions like MIT. In that field, trends have come and gone: Although different styles—*syntactic*,⁴⁶ *semantic*,⁴⁷ *statistical*,⁴⁸ *stochastic*⁴⁹—were popular at different times, all continued to develop in parallel. Even now as the latest large language models (LLMs) have amazed observers with the seeming intelligence of text output, the actual power of contemporary systems like OpenAI’s ChatGPT (generative pretrained transformer) are disguised behind their chatbot interfaces, which retain a resemblance to Weizenbaum’s original. These enticing facades obfuscate the machinery that often includes a combination of statistical predictions, rule-based procedures, and human labor disguised as machine labor. From the perspective of the user, this leaves limited opportunity to distinguish hype from substance, to understand how each system operates, and to see why it produces certain outputs.

Weizenbaum warned of the many issues this obfuscation could create. It can, for example, lead to exploitation as humans are replaced, harmed, or treated unfairly by these systems. As Weizenbaum says, “an individual is dehumanized whenever he is treated as less than a whole person. The various forms of human and social engineering ... do just that, in that they circumvent human contexts, especially those that gave real meaning to human language.”⁵⁰ Weizenbaum argued that removing language from its social contexts and treating it as a set of abstract concepts in a computational system can be dehumanizing. It risks ignoring the multiple meanings inherent to language, which are impossible to capture fully in AI systems and which can result in direct harm, rights violations, privacy breaches, exploitation, displacement, and discrimination.⁵¹ That is why it is essential to consider broader ethical and social impacts when designing, deploying, and using automated systems.

Emerging large language models support a new class of knowledge-powered systems.⁵² Under the surface, the chatbot interface runs on social labor. Its machinery supported by literally millions of traces of humans’ writings and conversations, siphoned into datasets usually without creators’ awareness or consent. That labor enables automated

cultural production that is managed, controlled, monitored, disaggregated, and reaggregated on demand. This abstracts human cultural production through software, treating it as a *standing reserve* for a computational system (in a similar fashion to electricity or a water supply, yet privatized and monopolized).⁵³ As Langdon Winner observed in 1978, it is possible “for inanimate instruments to perform their own work ‘at the word of a command or by intelligent anticipation,’ that is, by a computer program. This development has led to conjecture that the perfection of industrial technology will eventually liberate mankind from toil,”⁵⁴ a common story that OpenAI, Anthropic and other AI companies repeat today. Yet computation is always serviced by human labor—even as chatbots currently manage customer queries and problems, help with homework and replace teaching assistants, converse as companions and counselors, and entertain and entangle with notions of identity and human exceptionalism.⁵⁵ They also help produce a deluge of AI slop that is cannibalizing its source materials and the planet’s natural resources.⁵⁶ The idea that ELIZA might inspire computational systems that create cognitive factories and AI slop would likely have horrified Weizenbaum. It is, in a sense, the realization of cybernetic systems that attempt to closely couple machines and humans into tightly linked feedback loops—exactly the threat that Weizenbaum warned about in *Computer Power and Human Reason*.

In that book and other post-ELIZA writings, Weizenbaum speaks out as an early critic of what we now call the tech sector and its mindset of exponential acceleration, regardless of the exploitative relationships it may be encoding and the social and political consequences of abstracting computational systems. His own relationship to technology shifted as he saw the impact of ELIZA and the ways automated systems were being scaled up and leveraged for power, trends that have continued with a wider public fascination and unease with the emergence of “intelligent” machines. This book is a call to revisit Weizenbaum’s work, and its warnings, through direct engagement with the code and scripts of ELIZA itself, as well as through Weizenbaum’s writing and the works of others it influenced.

01.04 Re-Creating ELIZA

Those who wish to talk with DOCTOR directly can find an accurate reproduction of ELIZA/DOCTOR, created by Anthony (Ant) Hay, on the inventingeliza.com website.⁵⁷ This reproduction is based on analysis of the source code and includes the “certain counting mechanism” that allows it to replicate Weizenbaum’s ELIZA more accurately.

As part of our software archaeology work, we have also assembled two other notable recreations, one displayed in Paris and another available online on a CTSS simulator. In April 2025 at the Musée Jeu de Paume, museum visitors encountered a realistic simulation of what it might have been like using ELIZA in the mid-1960s. Ant and his son, Max,

translated Ant's C++ ELIZA to JavaScript. Jeff Shrager and his son, Leo, created a web-based simulation of Ant and Max's ELIZA running on an ASR-33 teletype, including image, sound, and timing that give a good sense of what it was like to run ELIZA in its original environment.⁵⁸

That recreation led to resuscitating the original ELIZA code on an IBM 7094 emulator. David M. Berry asked Rupert Lane, an aficionado of early operating systems, whether he could run the MAD-SLIP ELIZA code on an emulated version of the Michigan Time-Sharing System (MTS) (where the Michigan Algorithm Decoder (MAD) language that Weizenbaum used to implement ELIZA was developed).⁵⁹ It turned out to be easier, and more authentic, to rebuild MIT's CTSS operating system on the 7094 emulator, then reconstruct SLIP, MAD, and finally ELIZA on that stack.⁶⁰ Bringing the original ELIZA up on the MAD/CTSS stack required numerous steps of code cleaning, completion, and debugging.⁶¹ Some required functions were missing and had to be rewritten, but 96 percent of the running ELIZA MAD-SLIP code is running exactly as it was found in Weizenbaum's archive. The entire stack is open so that any user of a Unix-like operating system can run an authentic ELIZA on their version of a pioneering time-sharing system.⁶²

We see these reconstructions as part of our ongoing work to help contemporary audiences experience what it was like to talk with the first chatbot. The process of bringing ELIZA back has also helped us to develop a fuller understanding of the software in its original material context, as well as to better grasp its sociotechnical impact on this moment.⁶³

01.05 About This Book

Inventing ELIZA has been written by an interdisciplinary team of scholars, artists, philosophers, and programmers, who each inhabit different combinations of these roles. As a collection of investigators with diverse interests, we have constructed this book together, while also highlighting our individual perspectives that made this conversation so rich. Given our individual relationships to ELIZA, its history, and its legacy, certain chapters draw more on one or more authors' experiences, and we have indicated who they are. Of course, we do not always agree or speak with a single voice, so you will find occasional reference to each of us by our names or initials—but we hope this makes reading more interesting.

As part of the Software Studies series, this book uses the methods of critical code studies to explore and interpret the ELIZA code and the DOCTOR script. Critical code studies (CCS) is a set of emerging methods to explore the extra-functional significance of computer source code.⁶⁴ *Extra-* here means "growing out from," so first we make clear how the code and scripts operate. However, to understand the code, we need to establish the historical and material context in which it circulated and operated. Once that foundation is laid, we can discuss the implications and further meaning of this code in terms of culture, computer and human history, power dynamics, creative computing, automated therapy, and

the further legacy of chatbots including their use in AI. In other words, “a close reading of code can also draw attention to the way in which code may encode particular values and norms.”⁶⁵ Nonetheless, code here is not treated as a singular, static text but as a snapshot of a broader system at work. Code is both sign and cause of dynamic processes brought into action on hardware and interacting with other software in a field of shifting states. For this reason, our efforts to get the code running and to run other adaptations and approximations are crucial to our reading. The code is the entry point but also the grounding of our investigation. It is the artifact around which we gather to discuss and interpret. However, “the way in which code is actually ‘doing’ is vitally important, and we can only understand this by actually reading the code itself and watching how it operates.”⁶⁶

In this collaborative book, our methods are radically interdisciplinary, developed out of years of conversations among a team with expertise on computer programming, history, literary studies, philosophy, and art. As such, this book adapts its methods from previous collaborative close readings of software including *10 PRINT CHR\$(205.5+RND(1)); : GOTO 10* and *Reading Project*.⁶⁷ In our analysis of ELIZA and its impacts, we apply theories from across computational media studies (in particular software and platform studies, and media archeology).⁶⁸ We also draw on critical AI research (broadly defined, including critical algorithm and data studies, and histories of science and technology).⁶⁹ Many of these fields overlap, which makes sense given the rapid, widespread deployment of emerging technologies. There are many entry points for reading more on these topics, including the excellent anthologies such as *Your Computer Is on Fire* and *Uncertain Archives: Critical Keywords for Big Data*.⁷⁰

To illustrate the way code itself engenders conversation, this book is grounded in close readings of the ELIZA program code written in MAD-SLIP (chapter 6) and the DOCTOR script (chapter 7), both of which we have annotated collectively and individually for over three years. We use our understanding of the code as the foundation for the entire project.⁷¹ In studying a sophisticated piece of software such as ELIZA, it is inevitable that we will need to explain some complex technical concepts and ideas in detail throughout the book. We do our best to approach these explanations with enough specificity to satisfy the experienced but enough clarity not to intimidate the newcomer. This reading continues the work begun in several Critical Code Studies Working Groups (particularly 2016 and 2022).⁷² The goal of our annotation is not to offer a final, definitive set of interpretations of this code but instead a foundation and model for future discussions. These readings benefit from the diversity of our approaches and backgrounds and will only become richer with an even wider range of readers who become literate in this code. We hope this book serves as an invitation to explore further one of the most widely known, influential, and misunderstood pieces of software.

As such, the book offers many avenues of entry. We have composed all chapters iteratively and conversationally rather than

sequentially in isolation. We invite you to read and cross-reference in any order you choose.

Chapter 2 looks at Weizenbaum's life from his role as ELIZA's creator to prominent AI critic. Through historical context and biographical details, we examine how his early experiences shaped his growing concern that computation and artificial intelligence threatened to reduce human judgment to mere calculation. The chapter explores his intellectual evolution and his principled stand against what he saw as technology's encroachment on human reason.

Chapter 3 explains how ELIZA works. We complement Weizenbaum's 1966 *CACM* paper (reproduced in Appendix II) by giving both a high-level overview of ELIZA's operation and some low-level details from the source code that were not present in Weizenbaum's description.

Chapter 4 contextualizes the programming and technological challenges Weizenbaum faced and makes tangible the computing environment at the time ELIZA was written—including physical practicalities from punch cards to magnetic tape storage. We discuss how Weizenbaum's work on SLIP and ELIZA overcome the obstacles presented by the era's material constraints, as well as describe some of the material conditions of interacting with the system via teletype.

Chapter 5 compares a selection of ELIZA's multiple scripts and dialogues, extending beyond DOCTOR and revealing the program's lesser-known applications, from teaching physics to analyzing poetry. Through archival transcripts, we uncover how users engaged with ELIZA across various contexts, including a clinical psychology trial.

In chapters 6 and 7, we reproduce our discovery of the code for ELIZA and the script of DOCTOR (both annotated). These chapters not only offer insight into the working of the code and script but also demonstrate some of the thinking inspired by our close reading through critical code studies.

Chapter 8 reviews many of the offshoots of ELIZA, from its contemporaries to later successors, demonstrating ELIZA's long legacy in software. Rather than a comprehensive catalogue, we contextualize a sampling of bots most in dialogue with ELIZA.

Chapter 9 looks at a program inspired by ELIZA and discusses the continuing impact of ELIZA on our cultural conceptions of self: Peggy Weil's chatbot, MrMind, performed "The Blurring Test," chatting with people about what makes them human.

Chapter 10 takes up ELIZA's influence in education, featuring ELIZA's teaching scripts as well as Mavis Beacon Teaches Typing and Peter Millican's Elizabeth as case studies before exploring later uses of ELIZA in both formal and informal learning contexts.

Chapter 11 looks at the processes behind contemporary large language models, discussing how they build on or diverge from the techniques of ELIZA. We explore the implications of both their technical and their social aspects given what we have learned from ELIZA's code. Though vastly different from ELIZA, conversational models such

as ChatGPT still rely upon many of Weizenbaum's approaches, including demonstrating the deceptive "magic" of mechanized conversational exchange.

Finally, the conclusion reflects further on the implications of this research project, particularly at this moment in AI history.

Inventing ELIZA is also interspersed with short dialogues between humans and chatbots, both real and imaginary. These include selections from ELIZA/DOCTOR plus other previously unseen ELIZA scripts. They also include later adaptations of ELIZA, as well as chatbots that respond to the ELIZA form and the current chatbot boom, like Claire James Carroll's "The Imitation Game." These interludes are a glimpse into the dialogic heart of bots, the wordplay, the hits and misfires of these systems. In a book about chatting machines, they illustrate the power of chat.

We invite readers of this book to join us in our exploration of ELIZA, to read the code and the paratexts, and then to continue the conversation about ELIZA with us at our websites inventingeliza.com and elizagen.org.

Programs have a life of their own, and we piece them together with snapshots of code and sample outputs. The program is not the code alone, though any given instance of code can teach us much about a particular moment in its development. The code of any program is always partial. The program is more than the sum of instances of the code and its outputs over time, moving through many ecosystems and across many platforms. Software programs—like any media objects, like any stories—are ongoing, dynamic, versioned, and evolving texts, rather than fixed works.⁷³ Weizenbaum's work has been often reproduced and taught, read and analyzed, and ported into many other languages and platforms and has inspired countless descendants. We have collected and analyzed not only code but transcripts and scribbles, tangents and offshoots—knitting together a complex story among the numerous voices that make up ELIZA's history and possible futures. In our search for ELIZA, we still have many mysteries to explore, much code to uncover, read, and interpret. Weizenbaum framed our interactions with this dynamic code as conversations. Like the code and the story of ELIZA itself, we consider these conversations to be in process and ongoing, and we invite you to join us in the chat.

In the words of ELIZA, "Please go on."

Table 1.2

Script
Genealogy

This is an attempt to document the different scripts that are extant within the archive, mentioned in articles, or referenced elsewhere. We are sure there are many others.

#	FILENAME	Description	Type	Referenced	Archive	Year
1	1965 CONVER- SATION THE FIRST	In which the editor talks to the computer. Author: Andrew T. Weil.	OUTPUT	<i>Harvard Review</i> (Spring 1965) ⁷⁴	Published	1965
2	1965 CONVER- SATION THE SECOND	In which a distraught young lady consults the machine. Author: Weizenbaum.	OUTPUT	<i>Harvard Review</i> (Spring 1965)	Published	1965
3	1965 CONVER- SATION THE THIRD	In which a human psychiatrist meets his mechanical counterpart. Author: Weizenbaum.	OUTPUT	<i>Harvard Review</i> (Spring 1965)	Published	1965
4	ANTIPR		NONE	Taylor, "Automated Tutoring and Its Discontents" (WATSNU)		1968
5	ARITHM	Undertakes mathematical calculations within the conversation and is able to evaluate and return the result	SCRIPT, OUTPUT	Archive	Yes	Unknown
6	CANVEC	By this time the student has studied 4-vectors using a textbook of his choice. Notice that the machine is ready to analyze an example provided by the student (although in this conversation the student asks the machine to provide the example). Notice also that toward the end of the interchange the machine misinterprets the student answer "Wwhy not" to mean "No," thus ending the conversation.	OUTPUT	Taylor, "Automated Tutoring and Its Discontents"		1968
7	COLOR	Script to ask about colors of things	SCRIPT, OUTPUT	Archive	Yes	1967
8	COMMNT	"HINTS ON USING, CONTROLLING, AND COMPLAINING ABOUT ELIZA" May be built in, not script	NONE	Taylor, "Automated Tutoring and Its Discontents" (WATSNU script)		1968
9	DOC		NONE		No	December 18, 1967
10	DOCTOR	The famous Rogerian DOCTOR script	SCRIPT, OUTPUT	"Eliza" the CACM 1966 paper	Yes	1966
11	ELEVTR	Discusses the physics of an elevator	OUTPUT	Taylor, "Automated Tutoring and Its Discontents" (WATSNU)	Yes?	1968

12	F29	Appears to be an early version of the FIGURE mathematical definition script	SCRIPT	Archive	Yes	December 26, 1967
13	FIGURE	Appears to be a late version of F29 extended with many more definitions and responses	SCRIPT	Archive	Yes	January 9, 1968
14	FORVEC		NONE	Archive	No	1967
15	FRANCE		OUTPUT	Taylor, "ELIZA Program"	No	1967
16	FVP1	Discussion of the exercise FVP1, in which the student fills in a table and then discusses his table entries with ELIZA. The statement of the problem precedes the conversation. Notice that during the conversation (regarding question 3) the machine treats a wrong answer as right, but by accident the error is discovered by the student. Notice also that after the problem has been discussed to the satisfaction of the student (parts of the conversation omitted here), the machine initiates a discussion on some issues related to the problem.	OUTPUT	Taylor, "Automated Tutoring and Its Discontents"	No	1968
17	FVQUIZ		NONE	Taylor, "Automated Tutoring and Its Discontents" (WATSNU)		1968
18	GIRL	Written to demonstrate how to write scripts. A real tutorial script must be able, among other things, to follow a developing line of argument. The ELIZA scriptwriter has approximately 100 commands ("functions") available for instructing the computer how to proceed. GIRL uses only three of these functions: TYPE, GOTO, and POPTOP, the last two of which simply tell the machine how to begin processing the program section of the script.	SCRIPT, OUTPUT	Taylor, "Automated Tutoring and Its Discontents"	No	1968
19	HELP	May be a test script run by Weizenbaum	OUTPUT			Unknown
20	HOWCON	"HINTS ON USING, CONTROLLING, AND COMPLAINING ABOUT ELIZA" May be built in, not script	NONE	Taylor, "Automated Tutoring and Its Discontents" (WATSNU script)		1968
21	HOWUSE	"HINTS ON USING, CONTROLLING, AND COMPLAINING ABOUT ELIZA" May be built in, not script	NONE	Taylor, "Automated Tutoring and Its Discontents" (WATSNU script)		1968
22	INTRVW	Part of a student conversation using INTRVW, the script that begins the study of 4-vectors. Notice how the conversation proceeds even when the student expresses himself less than clearly. The symbol QQ is sometimes used instead of the question mark. Other times the question mark is simply omitted.	OUTPUT	Taylor, "Automated Tutoring and Its Discontents"	No	1968

23	MIT		NONE	Archive	No	1967
24	NEWENG	Discusses New England states	SCRIPT, OUTPUT	Taylor, “The ELIZA Program”	No	1967
25	ORTH1		NONE	Archive	No	1967
26	PHOTON		NONE	Archive	No	1967
27	POLETA	Discusses the “Pole and Barn Paradox”	OUTPUT		Yes?	Unknown
28	QMPROB		NONE	Archive	No	1967
29	RLPOLN		NONE	Archive	No	1967
30	SOS	“HINTS ON USING, CONTROLLING, AND COMPLAINING ABOUT ELIZA” May be built in, not script	NONE	Taylor, “Automated Tutoring and Its Discontents” (WATSNU script)		1968
31	SPACKS	Discusses a quoted line of poetry by Barry Spacks of the MIT humanities department	OUTPUT	Taylor, “Automated Tutoring and Its Discontents”	No	1968
32	STATES		NONE	Archive	No	1967
33	SYNCTA	Discusses time synchronization	OUTPUT	Taylor, “Automated Tutoring and Its Discontents” (WATSNU script)	Yes?	1968
34	WATSNU, PREVI- OUSLY LSTSCR	Lists all other scripts This may not be a script but instead actually a function in ELIZA	OUTPUT	Taylor, “Automated Tutoring and Its Discontents”		1968
35	XYPOLN		NONE			Unknown

This is a section of [doi:10.7551/mitpress/15790.001.0001](https://doi.org/10.7551/mitpress/15790.001.0001)

Inventing ELIZA

How the First Chatbot Shaped the Future of AI

By: Sarah Ciston, David M. Berry, Anthony C. Hay,
Mark C. Marino, Peter Millican, Jeff Shrager,
Arthur I. Schwarz, Peggy Weil

Citation:

Inventing ELIZA: How the First Chatbot Shaped the Future of AI

By: Sarah Ciston, David M. Berry, Anthony C. Hay, Mark C. Marino,
Peter Millican, Jeff Shrager, Arthur I. Schwarz, Peggy Weil

DOI: 10.7551/mitpress/15790.001.0001

ISBN (electronic): 9780262052504

Publisher: The MIT Press

Published: 2026



The MIT Press

The MIT Press
Massachusetts Institute of Technology
77 Massachusetts Avenue
Cambridge, MA 02139
mitpress.mit.edu

© 2026 Massachusetts Institute of Technology

All rights reserved. No part of this book may be used to train artificial intelligence systems or reproduced in any form by any electronic or mechanical means (including photocopying, recording, or information storage and retrieval) without permission in writing from the publisher.

The MIT Press would like to thank the anonymous peer reviewers who provided comments on drafts of this book. The generous work of academic experts is essential for establishing the authority and quality of our publications. We acknowledge with gratitude the contributions of these otherwise uncredited readers.

This book was designed by Stefanie Tam in Brooklyn, New York, and is set in Univers, Iosevka, and Courier.

Library of Congress Cataloging-in-Publication Data

Names: Ciston, Sarah author | Berry, David M. (David Michael) author | Hay, Anthony C. author | Marino, Mark C. author | Millican, P. J. R. author | Schwarz, Arthur I. author | Shrager, Jeff author | Weil, Peggy author
Title: Inventing ELIZA : how the first chatbot shaped the future of AI / Sarah Ciston, David M. Berry, Anthony C. Hay, Mark C. Marino, Peter Millican, Arthur I. Schwarz, Jeff Shrager and Peggy Weil.
Description: Cambridge, Massachusetts : The MIT Press, [2026] | Series: Software studies | Includes bibliographical references and index.
Identifiers: LCCN 2025028421 (print) | LCCN 2025028422 (ebook) | ISBN 9780262052481 paperback | ISBN 9780262052504 pdf | ISBN 9780262052498 epub
Subjects: LCSH: ELIZA (Computer program) | Weizenbaum, Joseph | Chatbots | Natural language processing (Computer science) | Artificial intelligence--History
Classification: LCC QA76.76.C52 C57 2026 (print) | LCC QA76.76.C52 (ebook)
LC record available at <https://lccn.loc.gov/2025028421>
LC ebook record available at <https://lccn.loc.gov/2025028422>

10 9 8 7 6 5 4 3 2 1

EU Authorised Representative: Easy
Access System Europe, Mustamäe tee 50,
10621 Tallinn, Estonia
Email: gpsr.requests@easproject.com